

Depth Audit — what's shallow and how to deepen it

Brutal evaluation of every implemented module: where it's a stub, where the math is correct but thin, where the architecture is missing entirely, and what the production-grade replacement looks like.

Numbers below: existing line count → proposed line count, with concrete additions.

1. Math kernels (`src/starknet_lvr/compute/`) — currently 1,050 lines, target ~3,500

`greeks.py` (115 → ~450 lines)

Currently shallow:

- Single position only — no portfolio of positions, no multi-tick profiles handled at the type level
- `vega` is mentioned in the docstring but **not implemented** (vega is zero for vanilla CL because it doesn't trade vol — but the *implied σ fee* has a vega worth computing)
- `delta` and `gamma` are scalars per call; no batched/vectorized variant for backtest performance (currently every backtest step calls `delta()` 30 times — easy 100× speedup)
- No higher-order Greeks: third derivative ("speed"), Greeks in token0 numéraire (we only have token1), Greeks-of-fees
- No multi-position aggregator: the right abstraction is `Portfolio = list[Position]` with vectorized aggregate Greeks
- No "Greeks at risk" — what's the dollar Greek per pool given current TVL?
- IL is computed against HODL but not against a passive USDC-only baseline; no IL term structure

Production-grade additions:

```
class Position:
    """existing - extend with batch, normalization, profile types"""

class Portfolio:
    """new - aggregate over many positions"""
    positions: list[Position]
    def delta(p): return sum(pos.delta(p) for pos in self.positions)
    def gamma(p): ...
    def vega_to_sigma_fee(p, sigma): ... # NEW

def greeks_vectorized(positions, prices) -> np.ndarray:
    """vectorized over price grid for backtest performance"""

def il_term_structure(pos, p_path) -> dict[horizon, IL]:
    """IL at 1d/7d/30d horizons given price path"""

def dollar_gamma(pos, p, tvl_usd) -> float:
    """USD-equivalent gamma at observed TVL - what an LP actually loses per  $\sigma^2$ """
```

`lvr.py` (92 → ~400 lines)

Currently shallow:

- Has the closed form $\sigma^2 \cdot L \cdot \sqrt{p} / 4$ and a TVL proxy. **Does not implement the Millionis empirical block-by-block LVR** — the truly informed-flow component, computed as $\sum (\Delta \text{inventory} \cdot (p_{\text{ref}} - p_{\text{pool}}))$ at each block where the price moves
- `lvr_integrated` uses trapezoidal — fine for smooth paths, broken for jump-y paths (which is what real

markets produce). Need both Riemann and explicit jump-decomposition (continuous + discrete LVR per Milionis 2024)

- No multi-pool aggregator; cannot compute portfolio LVR
- No realized-vs-expected LVR comparison (the "wedge" computed correctly is $\text{realized_LVR_per_window} - \text{fee_revenue_per_window}$, not the σ -implied form)
- Doesn't handle the case where the position goes out of range mid-window
- TVL proxy uses $k_{\text{eff}} = 1$ with no calibration; should expose k_{eff} from observed concentration distribution

Production-grade additions:

```
def realized_lvr_block_level(swaps, ref_prices, pool_state) -> float:
    """Empirical LVR: sum  $|\Delta \text{inventory} \cdot (p_{\text{ref}} - p_{\text{pool}})|$  at each crossing. THE actual quantity"""

def lvr_decomposition(pos, swaps, refs) -> dict:
    """Continuous + discrete + within-tick decomposition (Fukasawa et al. 2024)"""

def k_eff_from_distribution(pool_id, snapshot) -> float:
    """Calibrate the TVL-proxy concentration factor from observed position distribution"""

def lvr_with_extension_state(pos, p, sigma, ext_state) -> float:
    """When the pool has a dynamic-fee extension,  $\sigma$  enters via  $\phi$ """

def portfolio_lvr_rate(positions, p, sigma) -> float:
    """Multi-position aggregator"""

def lvr_attribution(pool_id, window) -> dict:
    """Decompose total LVR by (vintage  $\times$  strategy) — needed for cohort analysis"""
```

`sigma_fee.py` (118 $\rightarrow$ ~350 lines)

Currently shallow:

- Closed form for constant-fee case only. Real Ekubo pools with dynamic-fee extensions are a path-dependent regime where σ_{fee} depends on extension state ϕ — not handled
- Brent solver works but doesn't return convergence diagnostics (residual, iterations, sensitivity to bracket)
- No CI on σ_{fee} itself — when fees and TVL are noisy, σ_{fee} inherits a CI we should compute and propagate
- No batch term-structure call — currently `run_sigma_fee_term_structure.py` re-computes piecewise
- Doesn't differentiate σ_{fee} under different LVR conventions (Milionis instantaneous vs Cartea-Drissi-Monga continuous-time)

Production-grade additions:

```
def sigma_fee_with_ci(fees, L, p, dt, fee_se, tvl_se, n_resamples=2000) -> tuple[float, float, float]:
    """Bootstrap CI on  $\sigma_{\text{fee}}$  accounting for fee/TVL measurement uncertainty"""

def sigma_fee_term_structure_batch(pool_panel, windows) -> pd.DataFrame:
    """Vectorized term-structure across pools  $\times$  windows"""

def sigma_fee_with_extension_state(fees, L, p, dt, phi, ext_model) -> float:
    """Generic solver coupling extension-state evolution"""

def sigma_fee_decomposition(pool, window) -> dict:
    """Split  $\sigma_{\text{fee}}$  contribution: from informed flow vs noise vs JIT"""

def sigma_fee_under_funding(fees, L, p, dt, funding) -> float:
    """The right comparison if hedging on Extended:  $\sigma_{\text{fee}} - \text{funding-cost-equivalent}$ """
```

Currently shallow:

- Six standard estimators. **Missing**: realized variance with bias correction (Zhou 1996), two-scale realized vol (Zhang-Mykland-Aït-Sahalia 2005), kernel-based realized vol (Barndorff-Nielsen 2008), microstructure-noise filtering (TSRV)
- No jump-vs-continuous-vol decomposition (Barndorff-Nielsen-Shephard jump test)
- No volatility forecasting (HAR, GARCH, RV-AR) — important for σ_{fee} comparison since fees over the next window depend on *forecast* σ , not realized
- No autocorrelation diagnostics (Ljung-Box on returns and absolute returns)
- Single-asset only; no realized covariance (which we need for the cross-AMM comparison)

Production-grade additions:

```
def realized_vol_two_scale(prices, fast=1, slow=300) -> float:
    """Zhang-Mykland-Aït-Sahalia: corrects for microstructure noise"""

def jump_test_bnsh(returns) -> dict:
    """Jump-vs-continuous decomposition; return jump statistic, p-value, jump locations"""

def realized_covariance(returns_matrix) -> np.ndarray:
    """Multi-asset realized covariance (for portfolio-level risk)"""

def har_forecast(daily_rv, weekly_rv, monthly_rv, horizon) -> float:
    """HAR-RV forecast (Corsi 2009) — the  $\sigma$  to compare  $\sigma_{\text{fee}}$  against"""

def garch_forecast(returns, horizon, model="GARCH(1,1)") -> float:
    """alternative forecast"""

def vol_diagnostics(prices) -> dict:
    """LB test, ARCH test, normality of returns; informs which estimator to trust"""
```

Currently shallow:

- Linear interpolation of the reference; no spread-aware mid (bid-ask midpoint), no top-of-book vs depth-weighted
- Single-horizon; no horizon-conditional standard errors; no horizon-curve plotting
- No permanent vs temporary impact decomposition (Almgren-Chriss style)
- No swap-size-conditional markout (Kyle's λ — informed flow indicator)
- Doesn't aggregate by hour-of-day, day-of-week, swap-direction (these are the standard dashboards in any MM desk's adverse-selection report)

Production-grade additions:

```
def markout_with_se(swap, refs, horizon) -> tuple[float, float]:
    """Markout + standard error from quote-spread"""

def kyle_lambda(swaps, refs, window) -> float:
    """Linear price-impact coefficient — informed-flow proxy"""

def markout_attribution(panel) -> dict:
    """By hour, day, direction, size bucket"""

def permanent_vs_temporary_impact(swaps, refs) -> dict:
```

```

    """5min markout - 1min markout = permanent component"""

def markout_curve(swap, refs, horizons) -> dict:
    """Full term structure of markouts at 1s, 5s, 30s, 1m, 5m, 30m"""

```

lp_simulator.py (131 → ~600 lines)

Currently shallow:

- One profile (uniform-band centered on price); no concentrated-active, no real on-chain-reconstructed positions, no strategy abstraction
- One rebalance policy (no rebalance — band stays static); no rebalance-on-trigger, no scheduled rebalance, no Cartea-Jaimungal optimal-rebalance
- One swap-volume model (constant per step); no Poisson arrival, no toxic-flow contamination, no JIT competition
- No gas/slippage modeling for rebalances; no Ekubo-specific tick crossing costs
- No proper position lifecycle (open / extend / close / migrate); no fee-collection event modeling

Production-grade additions:

```

class Strategy(ABC):
    @abstractmethod
    def open(self, p, capital) -> Position: ...
    @abstractmethod
    def rebalance(self, current_pos, p, sigma) -> Position | None: ...

class UniformBandStrategy(Strategy): ...
class ConcentratedActiveStrategy(Strategy): ...
class RebalanceOnTriggerStrategy(Strategy): ... # rebalance when |Δ_drift| > threshold
class CarteaJaimungalOptimalRebalance(Strategy): ... # closed form

class SwapModel(ABC):
    """Volume + size + direction process"""

class PoissonSwapModel(SwapModel): ...
class ToxicFlowSwapModel(SwapModel): ... # adverse selection contamination
class JITCompetitionModel(SwapModel): ... # JIT searchers crowd in-range LPs out

class GasModel(ABC):
    def cost_of_swap(self, swap, network) -> float: ...

class StarknetGasModel(GasModel): ... # current Starknet cost model

class BacktestEngine:
    """Plug strategy + swap_model + gas_model; produces walk-forward attribution"""

```

bootstrap.py (108 → ~250 lines)

Currently shallow:

- Block bootstrap and cluster bootstrap. **Missing:** stationary bootstrap (Politis-Romano 1994 with random block lengths — *the* bootstrap for dependent series, more principled than fixed block); BCa intervals (bias-corrected accelerated, fixes block bootstrap's known bias); double bootstrap (CIs on CIs)
- No studentized statistic; no pivot CIs
- Block length always = $n^{\{1/3\}}$; no automatic selection (Politis-White rule, Hall-Horowitz-Jing)

Production-grade additions:

```
def stationary_bootstrap_ci(...): ...
def bca_ci(...):
    """Bias-corrected accelerated — addresses Block bootstrap's documented downward bias"""
def double_bootstrap_ci(..., n_outer=1000, n_inner=200): ...
def auto_block_length(series) -> int:
    """Politis-White (2004)"""
```

optimal_hedge.py (98 → ~600 lines)

Currently shallow:

- LQ closed-form approximation only. **The Bouchard et al. paper has an actual HJB PDE that should be solved numerically** for non-LQ cases
- Avellaneda-Stoikov skew is one stub function — should be a full inventory MM module
- No stochastic-control formalism: no value function, no optimal-stopping for entry/exit, no portfolio-of-vaults problem
- `funding_aware_hedge_ratio` is a heuristic — should be the actual constrained optimization

Production-grade additions:

```
class OptimalHedgeProblem:
    """Bouchard-Han-Hu-Sanchez-Betancourt setup — full state space"""
    def value_function_lq(self) -> Callable: ...
    def value_function_pde(self) -> Callable:
        """numerical HJB PDE on (t, q, p, basis) grid via finite differences"""
    def optimal_policy(self, state) -> Action: ...

class AvellanedaStoikovQuoter:
    """Full inventory-skewed MM quoter for the LP+perp pair"""
    def quote(self, mid, q, sigma, T_minus_t) -> tuple[bid, ask]: ...

class FundingAwareConstrainedHedge:
    """Solve the actual mean-variance with funding cost explicitly modeled"""

class CarteaJaimungalOptimalRebalance:
    """Closed-form solution for when to rebalance the LP band"""
```

squeeth.py (73 → ~300 lines)

Currently shallow:

- Constant-gamma assumption only. Real Squeeth has funding mechanism, oracle dependency, contango/backwardation behavior — none modeled
- Vega hedging not implemented (Squeeth has positive vega — useful for hedging the LP's negative vega)
- No comparison to actually-tradeable instruments on Starknet (Carmine options, perps with non-linear payoffs)

Production-grade additions:

```
class SqueethModel:
    def __init__(self, mark_curve, funding_curve, oracle_lag): ...
    def funding_rate_at(t, p, mark) -> float: ...
    def implied_vol_from_mark(p, mark) -> float: ...

class CarmineOptionsHedge:
    """Compare Squeeth to actual Cairo options on Carmine"""
    def hedge_lp_with_calls(pos, expiry) -> Portfolio: ...
```

```
class PowerPerpFamily:
    """ETH^p for any p; closed-form Greeks; comparison Squeeth vs Cubeeth"""
```

selectors.py (68 → ~200 lines)

Currently shallow:

- selector + felt encoding only. Missing: Cairo struct serialization helpers (encode/decode `PragmaPricesResponse`, `EkuboPosition`, `PoolKey`), enum variant tag handling, ABI parsing

Production-grade additions:

```
def encode_struct(name, fields) -> list[str]: ...
def decode_struct(name, calldata) -> dict: ...
def encode_enum(variant_index, payload) -> list[str]: ...

class CairoABI:
    """Parse a contract ABI JSON; produce typed call/decode helpers"""
```

2. Analysis (`src/starknet_lvr/analysis/`) — currently 332 lines, target ~1,500

hypothesis_tests.py (137 → ~600 lines)

Currently shallow:

- Newey-West HAC; Fama-MacBeth; 2SLS-IV; BH-FDR. **Missing the modern essentials:** Andrews (1991) optimal kernel/bandwidth, Driscoll-Kraay panel SE (the right SE for panel data with both cross-sectional and serial correlation), Conley spatial-correlation SE (relevant if pools cluster geographically/by sector), Hansen-Lunde realized-volatility model selection (for picking the σ estimator)
- 2SLS but no GMM with optimal weighting (overidentified case), no LIML, no first-stage diagnostics (F-stat, Stock-Yogo critical values for weak-instrument detection)
- No specification tests (Hausman, Sargan-Hansen J)
- BH-FDR but no Holm-Bonferroni, no Bonferroni-Hochberg, no positive-FDR variants

Production-grade additions:

```
class HACEstimator:
    """Andrews (1991) data-driven bandwidth + Bartlett, Parzen, QS kernels"""

def driscoll_kraay_se(panel_y, panel_x, lag) -> tuple: ...
def conley_se(y, x, distances, cutoff) -> tuple: ...
def hansen_lunde_model_confidence_set(estimators, criterion) -> set: ...

class IVEstimator:
    """LIML, k-class, GMM, all with first-stage F, Stock-Yogo, Hausman, Sargan-Hansen"""

class HypothesisTestResult:
    """proper dataclass: estimator, point, SE, t, p, CI (multiple methods), N, etc."""

def stock_yogo_critical(k1, alpha=0.05) -> float: ...
def sargan_hansen_j_test(residuals, instruments) -> dict: ...
```

cross_amm.py (72 → ~400 lines)

Currently shallow:

- Just a data class wrapping daily rows. **Doesn't actually run the cross-AMM regression** the proposal claims. No fixed-effects panel, no AMM-design dummies, no interaction terms, no decomposition of "what fraction of the wedge is AMM-design vs market-design"

Production-grade additions:

```
class CrossAMMPanel:
    """Properly aligned panel across Ekubo, Uniswap v3, Trader Joe LB"""
    def add_pool(self, amm, pool_id, observations: pd.DataFrame): ...
    def with_fixed_effects(self, dimensions=['amm', 'pool', 'date']) -> 'PanelEstimator': ...

class PanelEstimator:
    def fit(self, regressors, fe_dimensions) -> dict: ...
    def decomposition(self) -> dict:
        """% wedge variation explained by AMM-design vs market vs idiosyncratic"""

class AMMDesignFeatures:
    """Tick precision, fee schedule, hooks/extensions support, bin-discretization, etc."""
    def vector_for_amm(self, amm) -> np.ndarray: ...
```

cointegration.py (107 → ~400 lines)

Currently shallow:

- Engle-Granger two-step. **Missing:** Johansen multivariate test (for >2 assets, e.g., the BTC family WBTC/tBTC/LBTC together), error-correction model fit + half-life CI, Hansen instability test (does the cointegrating vector itself drift?), Phillips-Perron variants of ADF
- No diagnostics: KPSS test (the inverse — null is stationary), Jarque-Bera on residuals
- ADF p-value comes from statsmodels but no critical-value table for the case where regressors include a near-unit-root

Production-grade additions:

```
def johansen_test(y_matrix, det_order=0, k_ar_diff=1) -> dict: ...
def error_correction_model(y, x, beta_coint) -> dict:
    """ $\Delta y = \alpha(y - \beta x) + \Sigma \gamma \Delta y_{lag} + \epsilon$ """

def hansen_instability_test(y, x) -> dict: ...
def kpss_test(series, regression='c') -> dict: ...

class CointegrationDiagnostics:
    def adf(self): ...
    def pp(self): ...
    def kpss(self): ...
    def jarque_bera_residuals(self): ...
    def variance_ratio(self): ...
    def consistent_verdict(self) -> str: ...
```

3. Risk engine — currently 0 lines, target ~800

This is **entirely missing**. A real LP-economics paper has a risk section.

```
# src/starknet_lvr/risk/var.py
def historical_var(returns, alpha=0.05) -> float: ...
def gaussian_var(returns, alpha=0.05) -> float: ...
def cornish_fisher_var(returns, alpha=0.05) -> float: ...
```

```

def cvar_expected_shortfall(returns, alpha=0.05) -> float: ...

# src/starknet_lvr/risk/stress.py
class StressScenario(ABC):
    """ETH +50%, ETH -50%, peg break, vol spike, liquidity crisis"""
    def apply(self, position) -> float: ...

class PegBreakScenario(StressScenario): ...
class VolSpike(StressScenario): ...
class LiquidityCrisis(StressScenario): ...

# src/starknet_lvr/risk/correlation.py
def ledoit_wolf_shrinkage(returns_matrix) -> np.ndarray:
    """Shrinkage estimator – far better than sample covariance at small n"""

def factor_decomposition(returns_matrix, k_factors=3) -> dict: ...

# src/starknet_lvr/risk/attribution.py
def pnl_attribution_by_source(vault_history) -> dict:
    """Decompose realized P&L into fees, LVR, hedge, funding, basis, idiosyncratic"""

```

4. Backtest framework — currently 1 ad-hoc script per test, target proper engine ~600 lines

Currently shallow:

- `run_vault_backtest_v2.py` is 252 lines of ad-hoc orchestration. No reusable strategy abstraction, no walk-forward, no cross-validation, no parameter sweep.

Production-grade replacement:

```

# src/starknet_lvr/backtest/engine.py
class BacktestEngine:
    def __init__(self, strategy, market_data, gas_model, slippage_model): ...
    def run(self, period) -> BacktestResult: ...
    def walk_forward(self, train_window, test_window, step) -> list[BacktestResult]: ...
    def sweep(self, param_grid) -> pd.DataFrame: ...

# src/starknet_lvr/backtest/strategies.py
class DeltaNeutralVault(Strategy): ...
class SqueethGammaHedge(Strategy): ...
class LQOptimalHedge(Strategy): ...
class TriggerRebalanceStrategy(Strategy): ...
class CarteaJaimungalRebalance(Strategy): ...
class JITSquasherStrategy(Strategy): ... # JIT-aware passive LP

# src/starknet_lvr/backtest/metrics.py
class PerformanceMetrics:
    def sharpe(self, returns) -> float: ...
    def sortino(self, returns) -> float: ...
    def calmar(self, nav) -> float: ...
    def max_drawdown(self, nav) -> float: ...
    def time_under_water(self, nav) -> float: ...
    def turnover(self, positions) -> float: ...
    def hit_rate(self, returns) -> float: ...
    def tail_ratio(self, returns) -> float: ...

```

5. Indexer / data layer — currently scripts only, target proper service ~1,200 lines

Currently shallow:

- All scripts are one-shot pulls. No daemonized event indexer. No proper retry/backoff. No schema migrations. No dead-letter queue. No atomic writes.

Production-grade replacement:

```
src/starknet_lvr/indexer/  
├─ event_processor.py          # subscribes to RPC; processes events  
├─ normalizer.py              # raw event → canonical domain types  
├─ state_replay.py            # reconstruct pool snapshot per block  
├─ persistence.py             # TimescaleDB writer with idempotent upserts  
├─ pipelines/  
│   ├─ ekubo_swap.py  
│   ├─ ekubo_position_update.py  
│   ├─ pragma_oracle.py  
│   ├─ extended_perp.py  
│   └─ defispring_gauge.py  
└─ service.py                  # orchestrator daemon
```

Each component handles backpressure, supports replay-from-checkpoint, has structured logging.

6. Cairo library — currently 269 lines, target ~2,000

Currently shallow:

- Skeleton with `core::u256_sqrt` (loses 9+ digits of precision)
- 5 anchor-point tests; no fuzz tests, no property tests, no formal verification annotations
- No comprehensive ABI for downstream consumers (no `IEkuboGreeks` trait)
- No multi-position aggregator
- No oracle integration

Production-grade structure:

```
cairo/ekubo_greeks/  
├─ Scarb.toml                  # Alexandria Math + cubit deps  
├─ src/  
│   ├─ lib.cairo               # public API + traits  
│   ├─ interfaces.cairo        # IEkuboGreeks, IPortfolio, IRiskOracle  
│   └─ math/  
│       ├─ fixed_point_64x61.cairo # cubit-style 64.61 fixed-point  
│       ├─ newton_sqrt.cairo      # Newton-iteration sqrt with bounded error  
│       ├─ log_exp.cairo          # ln, exp via Taylor series  
│       └─ safe_arithmetic.cairo  
│   └─ position/  
│       ├─ position.cairo         # Position struct + methods  
│       ├─ portfolio.cairo        # multi-position aggregator  
│       └─ reconstruction.cairo    # build positions from on-chain events  
│   └─ greeks/  
│       ├─ delta.cairo  
│       ├─ gamma.cairo  
│       ├─ vega.cairo  
│       └─ higher_order.cairo
```


Currently shallow:

- Unit tests only. No property-based tests. No integration tests. No e2e. No benchmarks. No mutation testing.

Production-grade pyramid:

```
tests/
├─ unit/                # current – expand to ~150 tests with hypothesis
├─ property/            # property-based via hypothesis library
├─ integration/         # against staging Postgres + mock RPC
├─ e2e/                 # playwright on dashboard
├─ benchmarks/         # pytest-benchmark on math kernels
├─ load/                # locust on API rate limits
└─ mutation/           # mutmut score >75%
```

9. Observability + ops — currently nothing

Currently shallow:

- `print()` statements as logging. No metrics. No distributed tracing. No error reporting.

Production-grade:

```
# src/starknet_lvr/observability/
├─ logging.py           # structlog with JSON output
├─ metrics.py           # Prometheus client
├─ tracing.py           # OpenTelemetry
└─ errors.py            # Sentry integration

# Every service emits metrics:
# - indexer_events_processed_total
# - rpc_latency_seconds
# - sigma_fee_compute_duration
# - api_requests_total{endpoint, status}
```

10. Cross-cutting architectural problems

Problem A: No layered architecture

Currently `compute/`, `analysis/`, `api/` are sibling packages with cross-imports. The right shape is hexagonal:

- **domain/** — pure value objects (Position, Pool, Swap), no I/O, no framework
- **compute/** — pure math (depends on domain only)
- **infrastructure/** — RPC, DB, exchange API adapters
- **application/** — use cases (orchestrators)
- **interfaces/** — REST API, CLI, dashboard SSR

Problem B: No event-driven design

Indexer pulls. Compute is invoked from scripts. Dashboard would be a stub query layer. The right shape:

- Indexer → publishes Swap, PositionUpdate events to a queue
- Compute consumes events, materializes σ_{fee} / wedge / LVR per pool-day
- Dashboard subscribes to the materialized views (with SSE/WS for streaming)

Problem C: No distinction between business logic and orchestration

`run_vault_backtest_v2.py` mixes data loading, math, plotting, and CSV writing. The right shape: load → simulate → analyze → present, each as a separate layer.

Problem D: Reproducibility is fragile

`make repro` works *today* but several scripts depend on transient API responses. Need: versioned data snapshots (DVC, Pachyderm, or just timestamped CSVs in a `snapshots/` directory), pinned external-API contracts, deterministic seeds enforced.

Problem E: No multi-environment story

There's no notion of dev / staging / production. No env-var-driven config. Database is in `docker-compose` but no migration tooling.

Total target line count under the deeper architecture

Module	Current	Target	Δ
compute/	1,050	~3,500	+2,450
analysis/	332	~1,500	+1,168
risk/ (new)	0	~800	+800
backtest/ (new framework)	0	~600	+600
indexer/ (new)	0	~1,200	+1,200
domain/ (new)	0	~400	+400
infrastructure/ (new)	0	~600	+600
observability/ (new)	0	~300	+300
api/ (full)	81	~1,000	+919
Dashboard frontend (Next.js)	0	~2,500	+2,500
Cairo library	269	~2,000	+1,731
Tests (all levels)	521	~3,500	+2,979
Python + Cairo + TypeScript total	~2,500	~17,500	+15,000

That's roughly a **7× expansion**, organized into a clean layered architecture, with explicit risk and backtest frameworks, proper Cairo math, full dashboard, and a real testing pyramid. **This is what an institutional-grade DeFi research-and-product project actually looks like.**

Recommended phased implementation

Phase	What	Why first
P0 (week 1)	<code>domain/</code> + restructure <code>compute/</code> to depend on <code>domain</code> only	Foundational — every other module depends on it
P1 (weeks 2-3)	Production fixed-point Cairo + <code>ekubo-greeks</code> v0.5 with full Greeks, IL, LVR	The visible deliverable; SNF M1 gate
P2 (week 4)	<code>indexer/</code> event-driven service + TimescaleDB schema	Powers the dashboard

P3 (weeks 5-6)	<code>backtest/</code> framework + 6 strategies	Replaces <code>run_vault_backtest*.py</code> ad-hoc scripts
P4 (week 7)	<code>risk/</code> engine + <code>analysis/</code> deepening	Enables the empirical paper
P5 (weeks 8-9)	Dashboard frontend (Next.js)	SNF M2 gate
P6 (weeks 10-11)	Reference vault contract + audit prep	SNF M3 gate
P7 (week 12)	Observability, CI hardening, launch	M3 acceptance

Each phase is ~1-2 weeks of focused work, total ~12 weeks = exactly the M1+M2+M3 timeline.

What this earns you

- 1. Reviewers can verify everything.** The current repo has the *math* right; this expanded repo would have the *architecture* right too. SNF reviewers (and the partners you need at M2) read code, not just READMEs.
- 2. The empirical paper writes itself.** With proper indexer + risk engine + backtest framework, the paper becomes "we ran the framework, here are the findings" rather than "we wrote a paper with some scripts."
- 3. ekubo-greeks becomes adoptable.** A 2,000-line audited Cairo library with property tests and a reference oracle is what Re7 / Troves will actually integrate. The current 269-line skeleton is a demo.
- 4. The dashboard becomes a product.** Next.js + auth + streaming + embeddable widgets is what gets monthly active users; a FastAPI stub returning `{"note": "stub"}` does not.
- 5. The downstream commercial layer is plausibly fundable later.** A proper layered architecture with risk + backtest engines is one strategy contract away from a managed-vault product. The current scripts are not.

Want me to start P0 — restructure into `domain/` + cleaner `compute/` boundaries — or jump straight to P1 (production Cairo)?

--	--	--